

3.3 The Transformer Block

Positions, residual highways, a token-wise MLP, encoder versus decoder.

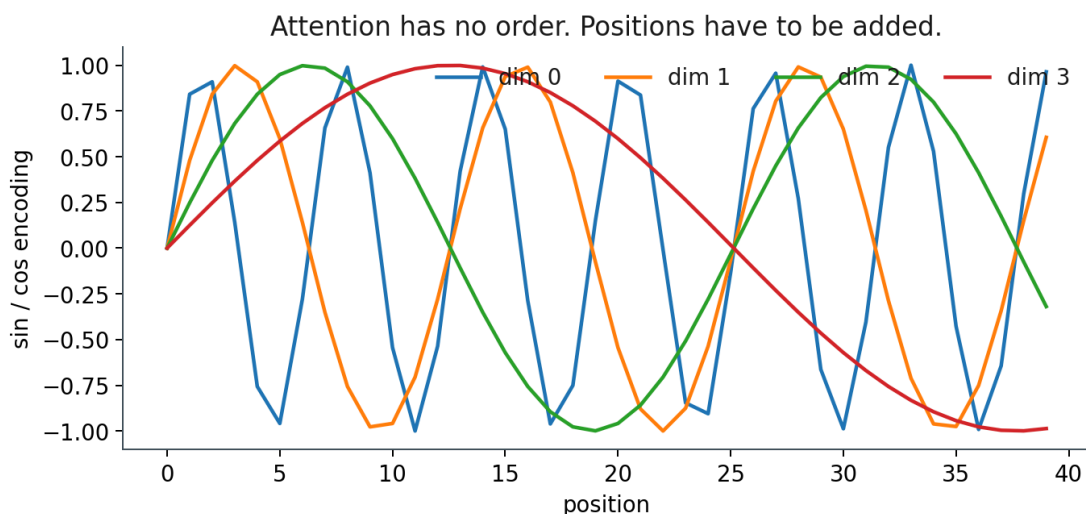
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A transformer **block** is attention, a residual add, layer norm, then a token-wise MLP, another residual and norm. Stack N of those. Attention mixes across positions; the MLP mixes across channels at one position. That is Princeton COS 484's assembly order.

1. Positions have to be added

Attention is a **set** operation: permute the tokens, the weights permute with them. Language is ordered.

Positional encodings (sinusoids in the original paper, or learned vectors) are added to token embeddings before the first block.

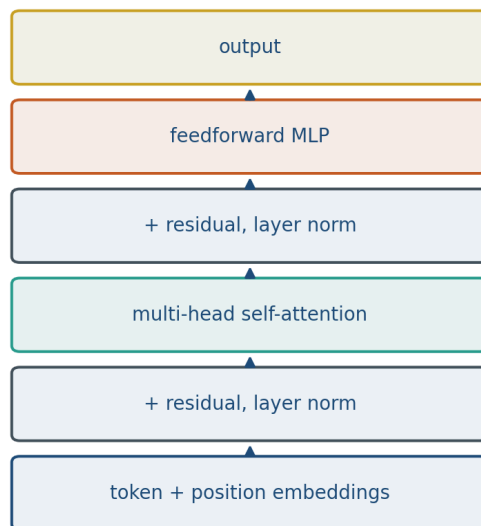


Relative or rotary positions (RoPE) appear in later LLMs. The idea is the same: put order into the vectors because the mixing layer will not. We do not derive RoPE this week.

Without positions, `dog bites man` and `man bites dog` can produce the same bag of contextual vectors up to permutation. That is fatal for language.

2. The block

One transformer block



Residual: add the input of the sublayer to its output. That is another highway, like the LSTM cell, so gradients can skip a messy attention map.

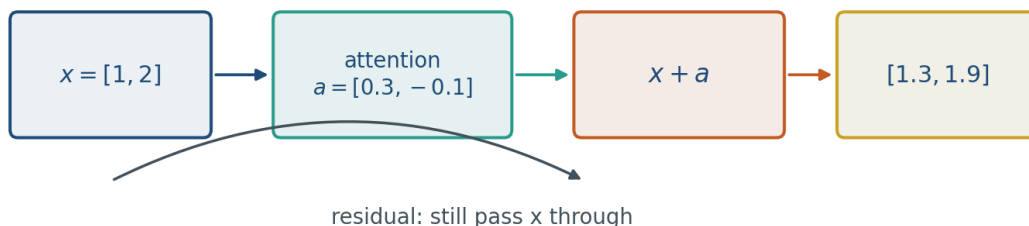
$$x \leftarrow x + \text{sublayer}(x).$$

Layer norm: normalize across features of **one token**. Pre-norm (layer-norm, then sublayer, then residual) is the usual modern default; it trains more stably at depth. Post-norm (sublayer, residual, then norm) is the 2017 paper. You will see `pre_norm` in Hugging Face configs. Do not swap them casually and call it an ablation of "the architecture."

MLP: usually two linear maps with a GELU, hidden width $4d$. Most of the parameters live here, not in attention.

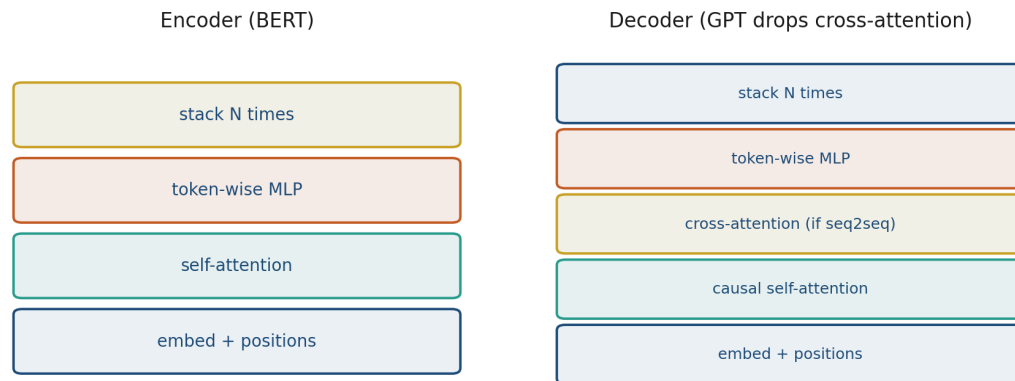
Count, roughly: attention's W_Q, W_K, W_V, W_O are four $d \times d$ maps $\approx 4d^2$. MLP is $d \times 4d$ and $4d \times d \approx 8d^2$. The MLP is heavier. Stack N blocks; GPT-2 small is $N = 12$, $d = 768$.

Add, do not replace. Same idea as the LSTM highway.



3. Encoder versus decoder

The original transformer has an **encoder** (bidirectional self-attention on the source) and a **decoder** (causal self-attention plus **cross-attention** into the encoder). Translation is the picture.



GPT keeps only a decoder (and drops cross-attention: there is no separate encoder). BERT keeps only an encoder. T5 and BART keep both; you do not need both unless the task is sequence-to-sequence.

Cross-attention: queries from the decoder, keys/values from the encoder. Captioning (BLIP, Week 5) uses that pattern with **image** tokens as the encoder side.

4. Teaching this note

~16 minutes. Positions first (one counterexample sentence), then walk the block with residuals as $x + \text{sublayer}(x)$, then the $4d^2$ vs $8d^2$ count, then encoder vs decoder as a two-column picture. Play Umar Jamil's **block walkthrough** (about 10:00–25:00). Skip the long from-scratch coding if this is a board day.

5. Worked example

Token vector $x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Attention sublayer emits $a = \begin{bmatrix} 0.3 \\ -0.1 \end{bmatrix}$. Residual:

$$x + a = \begin{bmatrix} 1.3 \\ 1.9 \end{bmatrix}.$$

If attention were zero (uniform mix that cancels, or a dead head), you still pass x through. That is the highway.

Parameter sketch: $d = 4$. Attention $\approx 4 \cdot 16 = 64$ weights (ignoring biases). MLP $\approx 8 \cdot 16 = 128$. The MLP has twice as many.

Positions: add $p_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ to token A and $p_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ to token B **before** the first block. Swapping tokens now swaps different vectors, not identical ones.

Layer-norm of $u = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$: mean 2, population std 1, so $(1, -1)$.

6. Where students get stuck

- Dropping positions and then being shocked that order vanished.
 - Thinking attention holds most of the parameters.
 - Residual as “replace x ” instead of “add to x .”
 - Drawing a GPT with encoder cross-attention “because Vaswani had it.”
-

7. Video

Watch [Umar Jamil: Transformer Neural Network — architecture and PyTorch](#).

Play the architecture / block walkthrough (roughly **10:00–25:00**). Pause on positional encodings and on the residual+norm sandwich. The hour of typing `nn.Linear` is optional homework, not this meeting.

The matching university lectures are Princeton COS 484 L8–L9 (encoder vs decoder stack) and CMU 11-711 (residual + layer norm + feed-forward as the remaining transformer parts). We do not copy those slides.

8. Practice

1. If you drop positional encodings, what two sentences become indistinguishable?
2. Residual connections help training. Which earlier idea in Week 2 is the cousin of that?
3. Where do most of the weights sit: W_Q, W_K, W_V, W_O or the MLP?
4. $d = 8$. Using the $4d^2$ vs $8d^2$ sketch, how many attention weights vs MLP weights (ignore biases)? What is the ratio MLP / attention?
5. Layer-norm a 2-D vector $u = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ by subtracting the mean and dividing by the standard deviation of its two coordinates (population std, no ε). Write the normalized vector.
6. Name the three sublayers of a Vaswani **decoder** block. Which one does GPT drop?
7. Pre-norm versus post-norm: which one is the 2017 paper, and which one do modern LLMs usually ship?